

요식업자로 살아남기

—

이
지
원

GitHub

github.com/JiwonEE2/DoriCooking

Notion

[Dori's Programming](#)

프로젝트 개요

게임 진행 플로우 차트

기능

구현

느낀점

목차

목표

자동 생산되는 요리를 손님에게 판매하고 빌런을 물리치며 식당을 성장시키는 게임

장르

2D 탑뷰 식당 경영 시뮬레이션 게임

개발 환경

Unity 2022.3.50f1
Visual Studio 2022

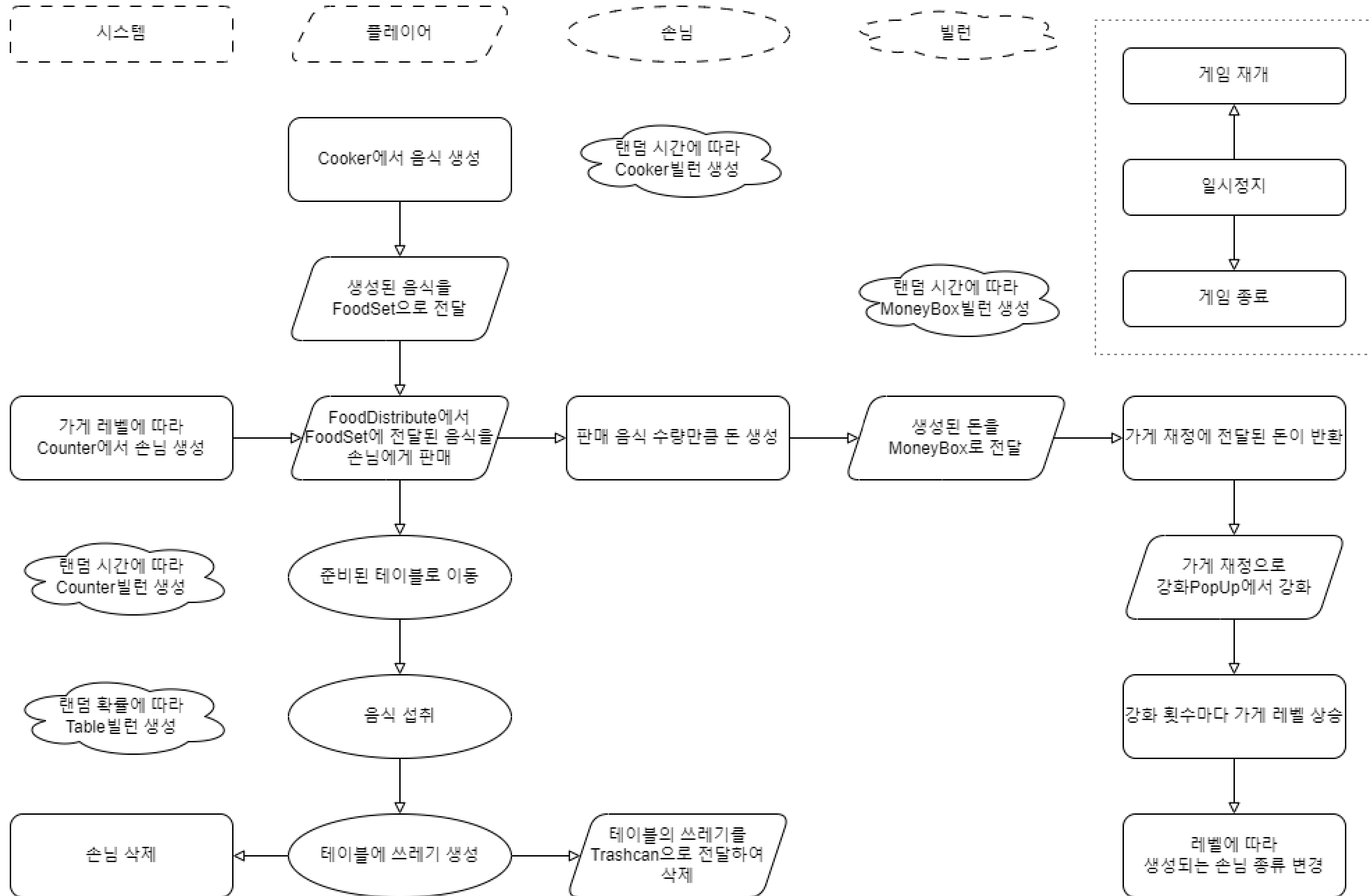
개발 인원

개발 1인, 기획 1인

개발 기간

2024.10.29 ~ 2024.11.08 (약 10일)

게임 진행 플로우 차트



플레이어

손님

빌런

소지가능 Object

상호작용 Object

추가적인 기능

기능

●●●●●

플레이어

이동 상하좌우 네 방향으로 이동	Object 소지 음식, 돈, 쓰레기 총 3가지의 오브젝트 소지 가능하며, 한 번에 한 종류의 오브젝트를 소지. 소지한 오브젝트의 종류는 씬에 표시	Object와 상호작용 Cooker, Counter 등의 여러 Object들과 상호작용
-----------------------------------	--	--

손님

종류

2종류의 손님이 존재하며, 가게 레벨에 따라 스폰 확률 변경

생성

카운터 앞에 생성되며 랜덤한 수량의 음식을 요청하고 대기

이동

음식을 받으면 비어있는 테이블을 탐색하여 해당 테이블로 이동

음식 섭취

음식의 수량에 비례한 시간 동안 음식 섭취 후 쓰레기 생성 및 삭제

빌런

CookerVillian

랜덤한 타이밍에 Cooker에 나타나
는 빌런으로 Cooker에서 플레이어
가 음식을 가져가는 것과 음식 생
성을 방해.

MoneyBoxVillian

랜덤한 타이밍에 MoneyBox에 나타
나는 빌런으로 MoneyBox와의 상호
작용(금고, 강화)를 방해하고, 30
초 내에 제거되지 못할 시 가게 재
정을 절반으로 줄이고 자동 삭제.
다른 빌런보다 삭제 대기 시간이
짧음.

CounterVillian

랜덤한 타이밍에 Counter에 나타나
는 빌런으로 Count와의 상호작용
(FoodSet, FoodDistribute, 손님 생
성, 돈 가져가기)을 방해.

TableVillian

손님 삭제 시 확률에 따라 해당
Table에 나타나는 빌런으로 플레이
어가 해당 Table의 쓰레기를 치우
는 것을 방해.

소지 가능 Object

Food

플레이어, 손님 이 소지 가능하며,
손님 생성 시 랜덤한 수량으로 요
청.

Money

플레이어가 소지 가능하며, 손님 이
요청한 음식의 수량만큼 Counter 좌
측에 생성

Trash

플레이어가 소지 가능하며, 손님 이
섭취한 음식의 수량만큼 테이블에
서 생성

상호작용 Object

Cooker

일정 주기로 음식이 생성

FoodSet

생성된 음식을 판매하기 전에
Counter에 올려두는 역할

MoneyBox

손님에 의해 생성된 돈을 넣어 사
용가능한 돈으로 변환

Counter

손님이 생성되는 위치이며 FoodSet
과 FoodDistribute를 통해 상호작용

FoodDistribute

Counter에 올려진 음식을 일정 시간
마다 판매

Trashcan

소지 가능한 Object를 종류에 상관
없이 삭제

추가적인 기능

일시정지 PopUp

ESC 키 입력을 받거나 우측 상단
일시정지 버튼 클릭을 통해 활성화
하며 키보드 입력을 통해 커서 이
동 및 엔터 키를 통한 행동 수행

강화 PopUp

키보드 입력으로 커서 이동 및 엔
터 키 입력 시 강화 조건(가게 재
정)에 부합할 시 강화

플레이어 손님 빌런 상호작용 Object(1) 상호작용 Object(2) Manager 및 Controller 추가적인 기능		11
	구현	

플레이어

이동

Moving() 선언을 통해 TimeScale이 1일 경우, 이동.

Input.GetAxisRaw와

Input.GetButtonDown/Up 을 통해 애니메이션과 함께 4방향으로 이동하도록 구현.

소지 Object 표시

enum ITEM = { MONEY, FOOD, TRASH, NONE } 를 선언하여 현재 아이템 종류를 저장.

가진 object가 0개 이하일 때, 개수 0, ITEM.NONE 으로 설정.

GottenItemShow() 선언을 통해 현재 아이템이 ITEM.NONE 이 아닐 때 말풍선과 함께 스프라이트 변경.

떨어진 돈 줍기

OnTriggerStay2D() 를 통해 플레이어의 아이템이 돈이거나 없을 경우가질 수 있는 만큼 가지고, 바닥의 돈을 삭제.

현재 아이템을 ITEM.MONEY로 설정

CustomerDataSO

확장성을 위해 손님의 데이터를 Scriptable Object로 생성.
이름, 요청최대/최소 음식 개수, 표시될 스프라이트로 구성.

CustomerContoller

손님의 생성 및 삭제 관리
CustomerSpawn() 선언을 통해 손님 생성. 손님 생성될 자리가 비어있고, 삭제된 지 1초가 지났을 경우 레벨에 따라 확률적으로 손님 생성
CustomerDestroy() 선언을 통해 손님 삭제. 손님 객체에서 삭제될 준비가 되었다는 신호를 받으면 해당 손님 삭제

충돌

OnCollisionEnter2D() 를 통해 충돌 발생 시 충돌 신호.
이동 중일 때만 이 신호로 정지.

음식 받기

FoodDistribute() 선언을 통해 음식 요구량이 부족한가?
Counter빌런이 없는가?
플레이어가 판매가능한 상태인가?
카운터에 음식이 있는가?
등 확인 후 수행.

준비된 테이블 탐색

음식을 받으면 준비된 테이블을 탐색하여 해당 손님의 테이블로 설정.

돈 내고 준비된 테이블로 이동

손님 생성 시간 초기화하고 기다리는 손님이 없음을 신호.
PayMoney() 선언을 통해 음식량만큼 돈 생성.
IEnumerator MovingCoroutine() 코루틴 선언을 통해 이동 구현.
이동 중 충돌 감지 시 1초 정지.

음식 섭취

먹을 음식이 남아있는가? 13
IEnumerator EatCoroutine() 코루틴 선언을 통해 일정 시간마다 음식 삭제 및 쓰레기 생성.

삭제

테이블에 쓰레기 표시.
현재 테이블을 쓰레기 테이블로 설정.
CustomerDestroy() 선언을 통해 Table빌런 생성을 위한 현재 테이블의 위치 반환 후 삭제 신호.

가진 음식 표시

FoodRender() 선언을 통해 테이블에 도착하기 전까지 손님의 몸에 음식 스프라이트 표시하고, 도착 후에는 테이블에 표시

VillianSpanwer

빌런들은 모두 VillianSpawner에서 생성되며, 빌런들의 재생성 시간과 Table 빌런의 생성 확률 설정 가능.

enum VILLIAN = { COOKER, COUNTER, MONEYBOX, TABLE } 선언을 통해 현재 생성된 빌런의 타입을 결정하여 해당 행동 수행.

Timer를 돌리면서 재생성 시간에 도달하면 Cooker,Counter,MoneyBox 빌런 셋 중 하나 랜덤으로 생성.

빌런 삭제 시 타이머 리셋.

Table 빌런의 경우, 손님 삭제 신호를 받으면 정해진 확률에 따라 Table빌런 생성.

Villian

4 종류의 빌런 구현을 위한 상위 클래스.

OnTriggerStay2D() 를 가상함수로 선언하여 플레이어가 영역에 들어와 있는 동안 Timer 시간에 따라 증가 후 지정해 둔 Time에 도달하면 삭제.

OnDisable() 을 가상함수로 선언하여 GameManager에 빌런이 없다고 신호. 이에 따라 UI에서 빌런 알림이 비활성화.

TableVillian

손님 삭제 시 확률에 따라 해당 Table에 나타나는 빌런으로 Villian 클래스를 상속.

손님 삭제 시의 테이블 번호를 받아 해당 손님이 있던 위치에 생성.

CookerVillian

랜덤한 타이밍에 Cooker에 나타나는 빌런으로 Villian 클래스를 상속.

두 Cooker의 상호작용 위치에 랜덤으로 생성.

CounterVillian

랜덤한 타이밍에 Counter에 나타나는 빌런으로 Villian 클래스를 상속.

손님이 있을 경우 다른 빌런이 생성.

MoneyBoxVillian

랜덤한 타이밍에 MoneyBox에 나타나는 빌런으로 Villian 클래스를 상속.

상호작용 Object(1)

Cooker

IEnumerator CookCoroutine() 선언을 통해 Cooker 빌런이 없을 때 일정 시간 간격으로 음식 생성.

FoodRender() 선언을 통해 재고가 1 이상일 때 음식 스프라이트 표시.

OnTriggerStay2D() 를 통해 Cooker 빌런이 없고, 플레이어가 상호작용 영역에 머무르고, 가진 아이템이 FOOD 이거나 NONE일 때 Cooker의 재고만큼 플레이어가 얻되, 플레이어가 가질 수 있는 만큼만 음식을 얻음.

FoodSet

OnTriggerStay2D() 를 통해 플레이어가 상호작용 영역에 머무를 때 플레이어의 현재 아이템이 FOOD이고, Counter빌런이 없을 경우 플레이어가 가진 만큼 넣되, 최대 재고를 넘지 않도록 구현.

FoodRender() 선언을 통해 재고가 1개 이상일 때 음식 스프라이트 표시

FoodDistribute

OnTriggerEnter2D() 를 통해 플레이어가 상호작용 영역에 들어왔고, Counter 빌런이 없을 경우 판매가능 신호를 true로 변경하고 판매 타이머 초기화.

OnTriggerExit2D() 를 통해 플레이어가 상호작용 영역에서 벗어났을 때 판매 가능 신호를 false로 변경. 강화 목록 중 자동판매 구현을 위해 bool 값을 선언.

상호작용 Object(2)

MoneyBox

OnTriggerEnter2D() 를 통해 플레이어가 상호작용 영역에 들어왔을 때 MoneyBox 빌런의 유무 확인 후 플레이어 가 가진 돈을 가게 재정으로 변환.

Enforce

OnTriggerEnter2D() 를 통해 플레이어가 상호작용 영역에 입장했을 때 강화창 활성화 및 TimeScale 을 0으로 설정.

Table

OnTriggerStay2D() 를 통해 플레이어가 상호작용 영역에 머무르고, Table 빌런이 없고, 플레이어의 아이템이 TRASH거나 NONE인 경우 쓰레기를 챙김.
쓰레기를 치우고, 손님이 할당되기 전 1초의 시간 지연을 위해 IEnumerator ReadyCheckCoroutine() 을 선언.

Trashcan

OnTriggerEnter2D() 를 통해 타이머 초기화.
OnTriggerStay2D() 를 통해 타이머 증가.
타이머가 일정 시간 경과하면 타이머를 초기화하고 플레이어의 아이템 삭제.

Manager 및 Controller

SingletonManager

다양한 Manager와 Controller를 Singleton으로 구현하기 위한 상위 클래스

GameManager

여러 위치에서 접근이 필요한 값들을 선언하고, 타이머를 실행.
레벨에 따른 손님 확률 정보, 타이머, 판매자동화 여부 등의 값이 존재.

SpriteManager

다양한 곳에서 참조할 Sprite를 할당하여 사용하기 위한 클래스

UIManager

빌런 생성 여부, 레벨, 소지금 등을 표시하고 일시정지 PopUp과 강화 PopUp을 관리.

강화 시 변경되는 값도 UIManager에서 선언되어 사용.

OnClickPauseButton() 선언을 통해 ESC 키 입력 외에도 버튼으로 일시정지 창 활성화 가능.

PauseGame() 선언을 통해

강화 PopUp이 활성화 상태면 비활성화, 비활성화 상태면 일시정지 PopUp을 활성화.

CustomerController

손님의 생성 및 삭제

TableController

준비된 테이블과 쓰레기가 있는 테이블을 리스트로 관리.

쓰레기 테이블에서 쓰레기가 없고, 준비가 되었다면 준비된 테이블로 변경.

준비된 테이블에서 쓰레기가 있다면 쓰레기 테이블로 변경.

추가적인 기능

PauseCursor

일시정지 PopUp이 활성화되었을 때 키보드로 입력을 받기 위한 클래스.

EnforceCursor

강화 PopUp이 활성화되었을 때 키보드로 입력을 받기 위한 클래스.
강화 가격 설정 가능.

GetKey() 선언을 통해 키 입력을 받고,
Move() 선언을 통해 커서 이동 구현.

기획서를 바탕으로 게임을 개발하는 것이 처음이고 짧은 마감 기간때문에 구조를 대충만 생각하고 진행하였습니다.

큰 그림을 보지 않고 아무거나 하나 씩 구현을 하니 뒤로 갈수록 예상치 못한 기능들을 추가해야 했고, 그때마다 구조가 크게크게 바뀌었습니다.

처음에는 빠르게 진행하였지만, 속도는 점점 느려지고 코드는 일관성이 없어졌습니다.

이에 프로젝트 시작 전 클래스 다이어그램, 플로우 차트 등을 귀찮더라도 꼭 만들어보고 진행하면 좋겠다고 뼈저리게 느꼈습니다.

감사합니다.

느낀 점

—